Build With Claude — Day 1

Building a Product with Claude Code: from idea to a running app

Author: Manaranjan Pradhan · www.manaranjanp.com

What you will build today

Ask DB is a web application in which business users ask questions of a database in plain English and get back a read-only SQL query, its result and an explanation. By the end of the day you will have:

- a one-page product idea and a full specification, both written by interviewing *you*;
- a project scaffold under Git with permissions, a `CLAUDE.md`, a skill, a hook and a subagent;
- an implementation plan in thin vertical slices, executed slice by slice by Claude Code;
- the app running locally against the sample CloudMetrics database;
- one GitHub issue fixed through a branch, tests and a pull request.

The day is organised as the sequence of steps you will execute. Each step has three parts: **what to do**, **why**, and **what you should see**. Steps marked ⌨ are typed at your shell; steps marked ▶ are typed inside Claude Code.

Before you start

Item	Requirement
Operating system	macOS, Linux, or Windows 10/11 (native PowerShell, or WSL2 if you prefer a Linux shell)
Accounts	A Claude account with Claude Code access; a GitHub account
Tools	`git` (on Windows: **Git for Windows**, which also provides the Git Bash that Claude Code uses to run shell commands and hooks), Node.js 18+ (for the front end), `uv` (installs Python for you), `jq`, the GitHub CLI `gh`
Files provided	`cloudmetrics.db` (SQLite database), `cloudmetrics.yaml` (schema and business rules), `cloudmetrics_eval.yaml` (evaluation set)

How to read the commands in this guide. Where a command differs between systems, two blocks are given: **macOS** (Terminal; also valid on Linux and WSL) and **PowerShell** (Windows). A single block means the command is identical on both. Commands typed inside Claude Code (slash commands, prompts, and `!git ...` shell-mode lines) are the same everywhere.

Install the small tools once. Open a new terminal afterwards so that `PATH` is reloaded.

macOS:

```
brew install git jq gh uv node
```

PowerShell (run as your normal user; `winget` ships with Windows 10/11):

```
winget install Git.Git
winget install jqlang.jq
winget install GitHub.cli
winget install astral-sh.uv
winget install OpenJS.NodeJS.LTS
```

Linux / WSL:

```
sudo apt install git jq gh nodejs npm && curl -LsSf https://astral.sh/uv/install.sh
| sh
```

Part 1 — Install and get to know Claude Code

Step 1 ⌨️ Install Claude Code

macOS:

```
curl -fsSL https://claude.ai/install.sh | bash
```

PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Either system, with Node.js already installed:

```
npm install -g @anthropic-ai/claude-code
```

Open a new terminal after installation so that `claude` is on your `PATH`. On Windows, Claude Code needs Git for Windows (installed above): it runs shell commands and hook scripts through Git Bash, so the `.sh` scripts later in this guide work unchanged.

Step 2 ⌨️ Check the version

```
claude --version
```

Why. Several commands used today (`/btw` , `/goal` , `/design` , `/advisor`) are recent. If your version is older than the one the instructor shows, run `claude update` .

Step 3 ▶ Log in

```
claude
```

The first run opens a browser window for authentication. Choose your Claude account (subscription) or an API key from the Claude Console, as your organisation directs. When the prompt `>` appears, you are in an interactive session.

Step 4 ▶ Try the basic commands

Type each of these and read what comes back. Slash commands are instructions to Claude Code itself, not to the model.

Command	What it does	What to look at
`/model`	Choose the model for this session	Pick the default for now. Opus-class models reason better on specs and plans; Sonnet-class models are faster for routine slices.
`/advisor`	Turns an advisor model on or off: a second, stronger model that reviews the main model's plan at key points	Useful during planning; costs extra tokens.
`/effort`	Sets how much reasoning the model spends per turn (`low` , `medium` , `high`)	Higher effort for specs and plans; lower for mechanical edits. See https://platform.claude.com/docs/en/build-with-claude/effort
`/statusline`	Configures the line at the bottom of the terminal (model, context used, cost, git branch)	Turn on context usage; you will watch it all day. See https://code.claude.com/docs/en/statusline
`/usage`	Shows your plan limits and how much of them this session has used	Check it at the end of each part.
`/help`	Lists every command	Skim it once.
`/exit`	Leaves Claude Code	`Ctrl+C` twice does the same.

Why. The commands you will use most are `/clear` (empty the context and start fresh), `/context` (see what is in the context), `/model` and `/effort` . The rest you need to know exist.

Part 2 — From idea to specification

Step 5 ⌨ Create a GitHub repository

On github.com create an **empty** repository named `askdb` (no README, no `.gitignore` , no licence; Claude Code will create them). Note the HTTPS URL, for example `https://github.com/<you>/askdb.git` .

Step 6 ⌨ Log in to GitHub from the terminal

```
gh auth login          # choose GitHub.com, HTTPS, authenticate in the browser
gh auth status         # should say "Logged in to github.com"
```

Why. `gh` gives both you and Claude Code a way to list issues and open pull requests from the command line. Git pushes over HTTPS will use the same credential.

Step 7 ⌨ Create the project folder and start Claude Code

macOS:

```
mkdir askdb && cd askdb
claude
```

PowerShell:

```
mkdir askdb; cd askdb
claude
```

Always start Claude Code **inside** the project folder. The folder is its working directory; `CLAUDE.md`, `.claude/` and the plan file are all found relative to it.

Step 8 — Read about the AskUserQuestion tool

Open https://code.claude.com/docs/en/tools-reference and find **AskUserQuestion**. It is the tool the model uses to ask you a structured, multiple-choice question and wait for your answer. The next two steps rely on it: instead of guessing what you want, Claude interviews you.

Step 9 ▶ Interview for the idea → `one-pager.md`

First empty the context:

```
/clear
```

Then paste this prompt:

```
I want to build a web app where business users ask questions of a database in
plain English. Before proposing anything, interview me using the
AskUserQuestion tool. Ask 4 or 5 questions, one at a time, about what the app
should do for its users: who they are, what they ask, what they need to see,
what must never happen, and what is out of scope. Do not ask about technology,
architecture or implementation. Then summarise what you heard.
```

Answer each question from the point of view of the business users you have in mind. Suggested answers for the course:

- **Who:** business analysts and support staff who know the business but not SQL.
- **What they ask:** counts, totals, trends and comparisons over customers, subscriptions, invoices and tickets; follow-up questions that refine the previous one.
- **What they need to see:** the answer, the SQL that produced it, a short explanation, and the row count; the ability to pick which database they are asking.
- **What must never happen:** any statement that modifies data; queries that run for minutes; leaking one database's data into another's session.
- **Out of scope:** charts, scheduled reports, write access, user management beyond simple roles.

When the summary appears and you agree with it:

```
Write the summary to one-pager.md.
```

Why. A one-pager written from your answers is the first artefact of the day, and everything after it refers back to it. Interviewing forces the model to stay on the problem and keeps technology out of the conversation until the problem is clear.

What you should see. `one-pager.md` in the project folder, half a page long, with no mention of frameworks or databases.

Step 10 ▶ Interview for the specification → `SPEC.md`

```
/clear
```

```
I want to build an application on the idea specified in one-pager.md. Interview me in detail
using the AskUserQuestion tool.

Ask about technical implementation, UI/UX, edge cases, concerns, and tradeoffs. Don't ask obvious
questions, dig into the hard parts I might not have considered.

Keep interviewing until we've covered everything, then write a complete spec to SPEC.md.
```

This interview is longer (ten to twenty questions). Decisions to make for the course build, so that everyone ends up with a compatible app:

Topic	Course answer
Back end	Python, FastAPI, SQLAlchemy; `uv` for packages
Front end	A small React (Vite) client; or plain HTML served by FastAPI if you prefer less tooling
Model access	Claude API (`anthropic` Python SDK); key from the `ANTHROPIC_API_KEY` environment variable
Databases	Any SQLAlchemy URL; SQLite for the course. Databases are added through an **admin page** (URL + schema YAML), not from files in the repository
Configuration storage	Each database saved as `<ASKDB_CONFIG_DIR>/databases/<name>.yaml`; validated, then written atomically; the directory is reloaded on restart
Safety	Read-only: only `SELECT` / `WITH`; a row cap and a timeout per query; the schema YAML's business rules are given to the model
Users and roles	A command-line `create-user`; roles `admin` (manages databases) and `support` / `analyst` (chat only)
Conversation	Follow-up questions keep the previous question and SQL as context
Explanations	Every answer shows the SQL and a two-sentence explanation
Clarification	When a question is ambiguous (for example, "revenue" with two possible definitions), ask instead of guessing
Non-data questions	Decline politely and say what the app can answer

After `SPEC.md` is written, read it. Ask for changes in plain language ("Section 4: the timeout is 5 seconds by default, configurable per database"). Finally:

```
Move one-pager.md and SPEC.md into a docs folder.
```

Why. The specification is the source of truth for the rest of the day and for Days 2 and 3. Time spent here is repaid in every slice. The "dig into the hard parts" instruction is what makes the interview worth doing; without it the model asks what you already know.

What you should see. `docs/one-pager.md` and `docs/SPEC.md`. The spec should have sections for users and roles, the query pipeline, safety rules, the admin page, the configuration format, the API, the UI, errors, testing, and an **Open items** list.

Step 11 ▶ Add the test data and the evaluation set to the spec

Copy the three provided files into the project from a second terminal (adjust the download folder):

macOS:

```
mkdir -p test && cp ~/Downloads/cloudmetrics.* test/
```

PowerShell:

```
New-Item -ItemType Directory -Force test | Out-Null
Copy-Item "$HOME\Downloads\cloudmetrics.*" test\
```

Then, in Claude Code:

```
Though the app will be data agnostic and any domain data can be configured, the initial SQLite
database, schema file and evaluation set are provided for testing the app during development. The
three files are:

test/cloudmetrics.db — the database
test/cloudmetrics.yaml — schema definition and business rules
test/cloudmetrics_eval.yaml — evaluation set

Add this information to docs/SPEC.md. Read the two YAML files and document their formats in the
spec as the formats the app must accept; adjust the formats only if something in them conflicts
with the spec, and tell me what you changed. Keep the files under the test folder.
```

Why. The schema YAML format becomes the format the admin page accepts, and the evaluation set becomes the acceptance test for the query pipeline. Documenting them in the spec means every later slice knows their shape. Day 2 writes the skills that generate these two files for a new database; Day 3 uses the same files for agent evaluation.

What you should see. A new section in `SPEC.md` describing the three files, the schema YAML format (tables, columns with descriptions, business rules, metric definitions) and the evaluation format (cases with a question, category, expected SQL or expected behaviour).

Step 12 ▶ Close the open items

```
Can we go through each of the open items in docs/SPEC.md one by one and interview me for
clarifications? Update the spec after each one and remove it from the open items list.
```

Why. A spec with open items produces slices with open items. Resolve them now while the whole picture is in context.

What you should see. An empty (or deliberately short) Open items section.

Step 13 ▶ Clear the context

```
/clear
```

The interview is over. Everything that matters is in `docs/`. From here on each task starts with a fresh context and reads the files it needs.

Part 3 — Project setup: memory, permissions, Git

Step 14 ▶ Generate the default files

```
/init
```

Claude Code looks at the folder and drafts a `CLAUDE.md`. Because the folder holds only `docs/` and `test/`, the draft will be thin. That is fine; the next step replaces it.

Step 15 ▶ Write `CLAUDE.md`

`CLAUDE.md` is loaded into the context at the start of **every** session. It holds standing rules: how the project is built, what the model must never do, and where to find the spec. Keep it short (under 60 lines); long files are read less carefully and cost context on every turn.

Ask Claude Code to write it:

```
Replace CLAUDE.md with project instructions for this repository, based on docs/SPEC.md. Include:
a two-line description; the stack and the commands to install, run, test and lint; the folder
layout; the guardrails (read-only SQL, row cap and timeout, never commit secrets, never read
.env); the rule that docs/SPEC.md is the source of truth and docs/PLAN.md tracks progress; and
the convention that every change comes with tests. Keep it under 60 lines.
```

A reference version is at the end of this guide (Appendix A). Compare and adjust.

Step 16 ▶ Check that `CLAUDE.md` is in the context

```
/context
```

The file was created during this session, so it is **not** in the context yet; `CLAUDE.md` is read when a session starts. Run:

```
/clear
/context all
```

What you should see. `CLAUDE.md` listed under **Memory files**, with its token count. Note the number; it is the fixed cost you pay on every turn.

Step 17 ▶ Prove the model is using it

```
What are the required guardrails for this project?
```

The answer should quote your guardrails (read-only SQL, row cap, timeout, no secrets). The model can only know them from `CLAUDE.md`; nothing else is in the context. This is the test you use whenever you doubt whether a memory file is loaded.

Step 18 ▶ Create `.claude/settings.json` with permissions

```
Create .claude/settings.json with this content:

{
  "permissions": {
    "allow": ["Edit", "Bash(pytest *)", "Bash(git add *)", "Bash(git commit *)"],
    "ask": ["Bash(git push *)"],
    "deny": ["Read(./.env)"]
  }
}
```

Why. Permissions decide what Claude Code may do without asking. The file above lets it edit files, run tests and make commits on its own (the loop you will run all afternoon), asks before every push, and refuses to read `.env` even if asked. `.claude/settings.json` is committed and shared with the team; personal overrides go in `.claude/settings.local.json`, which is not committed.

Step 19 ⌨ ▶ Initialise Git using shell mode

Inside Claude Code, a line that starts with `!` runs in your shell and the output is shown to you (and added to the context). Use it for quick checks without leaving the session.

```
!git init
!git status
```

What you should see. `Initialized empty Git repository`, then the untracked files: `CLAUDE.md`, `.claude/`, `docs/`, `test/`.

Step 20 ▶ Create the environment file template

```
Create .env.example with the key ANTHROPIC_API_KEY (value to be set later). Create .gitignore for
a Python and Node project and make sure .env is in it.
```

Why. The real key lives in `.env`, which is ignored by Git and denied to Claude Code by the permission rule above. `.env.example` tells a new developer which variables exist.

Step 21 ▶ First commit

```
!git add .
!git commit -m "initial scaffold for the project"
!git status
```

What you should see. One commit; `nothing to commit, working tree clean`.

Step 22 ▶ Connect the remote and push

```
!git remote add origin https://github.com/<you>/askdb.git
!git remote -v
!git branch -M main
!git push -u origin main
```

What you should see. `remote -v` shows `origin` twice (fetch and push). The push prints the branch tracking line. Refresh the repository page in the browser: the files are there.

If the push asks for a password, use `gh auth setup-git` once so that Git uses the `gh` credential.

Part 4 — Planning in thin vertical slices

Step 23 ▶ Enter plan mode

Press `Shift+Tab` until the status line reads **plan mode** (or type `/plan`). In plan mode Claude Code reads and reasons but does not write files or run commands that change anything. It is the right mode for designing before building.

Step 24 ▶ Clear the context

```
/clear
```

Plan mode survives `/clear`; check the status line.

Step 25 ▶ Create the plan

```
Read docs/SPEC.md. Propose an implementation plan as thin vertical slices.
Create 7–8 slices that can be implemented one after another. Each slice is end-to-end,
testable on its own, and enough to finish in one short session.

For each slice list: goal, files touched, how we verify it, and which
earlier slice it depends on. Do not write any code.
```

Claude Code may ask questions (it uses AskUserQuestion in plan mode too). Answer them. Read the proposed slices critically:

- **Vertical** means each slice goes from the API (or UI) down to the database and back, for one narrow capability. "Build the data layer" is a horizontal layer, not a slice.
- The first slice should be runnable within the hour: a FastAPI app with a health endpoint and one test.
- Each slice names how it is verified: a test command, a `curl`, or a click.

A typical result for Ask DB:

1. Project skeleton: FastAPI app, `uv` project, health endpoint, pytest, ruff.
2. Configuration registry: load `databases/*.yaml` from `ASKDB_CONFIG_DIR`; validate; atomic save.
3. Read-only SQL executor: `SELECT`-only guard, row cap, timeout, against `test/cloudmetrics.db`.
4. Question → SQL with the Claude API using the schema YAML; return SQL, rows, explanation.
5. Users and roles: `create-user` CLI, login, session.
6. Admin page: add a database by URL and uploaded YAML.

7. Chat UI: pick database, ask, see SQL and result, follow-up context.
8. Evaluation runner: run `test/cloudmetrics_eval.yaml` and report pass rate.

Step 26 ▶ Write the plan to `docs/PLAN.md`

Leave plan mode (`Shift+Tab` until the status line is back to normal). Then:

```
Write the agreed plan to docs/PLAN.md as a checklist: one checkbox per slice,
with its files and its verification method under it. Nothing else.
```

Why the format matters. The hook and the skill you add in Part 5 read this file. Each slice must be one line of the form `- [ ] **Slice N: goal**` with its details indented beneath. Check the file; if the format differs, ask for it to be changed.

Step 27 ▶ Review the slice sequence

```
Look at docs/PLAN.md. Are there slices that are too small and should be merged, or slices that
depend on each other so tightly that they must be done together? Propose a revised sequence,
explain each change in one line, and update docs/PLAN.md if I agree.
```

Why. The first plan is rarely right. Ten minutes of re-sequencing now saves an afternoon of half-finished slices.

Step 28 ▶ Commit and push

```
!git add docs/PLAN.md
!git commit -m "implementation plan in vertical slices"
!git push
```

The push triggers the **ask** permission: you approve it once.

Part 5 — A skill and a hook for the implementation loop

The afternoon's loop is: implement one slice, verify, commit, stop; review; clear; next slice. A **skill** encodes the procedure so that you type one command instead of a paragraph. A **hook** runs a script at a fixed moment (here: after every response) so that progress is always visible.

Step 29 ⌨ Create the folders

In your shell (second terminal, at the repository root). The hook uses `jq` to emit JSON, so check it is installed.

macOS:

```
mkdir -p .claude/hooks .claude/skills/next-slice .claude/agents
which jq || brew install jq
```

PowerShell:

```
New-Item -ItemType Directory -Force .claude\hooks, .claude\skills\next-slice,
.claude\agents | Out-Null
jq --version    # if not found: winget install jqlang.jq, then open a new terminal
```

Create the files in Steps 30, 31 and 37 with any editor (VS Code: `code .claude/skills/next-slice/SKILL.md`), or paste the content into Claude Code and ask it to write the file at that path. Save `.sh` files with LF line endings; on Windows, Git for Windows may convert them to CRLF, which breaks `#!/usr/bin/env bash`. To prevent that, run once in the repository: `git config core.autocrlf false`, and add a `.gitattributes` with the line `*.sh text eol=lf`.

Step 30 ⌨️ Create the skill

Save as `.claude/skills/next-slice/SKILL.md`:

```
---
name: next-slice
description: Implement the next unticked slice in docs/PLAN.md, verify it, commit,
record learnings, and stop.
---

Work on exactly one slice.

1. Open docs/PLAN.md. Find the first unticked slice. State its number and goal in
one line.
2. Read every file the slice lists before changing anything. Read any notes left
under the slice by earlier runs.
3. Implement the slice. Stay inside its scope; if something outside it needs
changing, note it under the relevant later slice in docs/PLAN.md instead of doing
it now.
4. Run the slice's verification steps, then the full suite:
   `uv run pytest -q && uv run ruff check .`
   Fix failures. Do not weaken or skip a test to get green.
5. If the slice cannot be finished in this session, split it into two entries in
docs/PLAN.md, finish the first, and stop after it.
6. Record what a future slice needs to know:
   - A design decision, constraint, or gotcha tied to a specific later slice: add a
one-line note under that slice in docs/PLAN.md.
   - Something about how this project works that is not derivable from the code (a
tool quirk, a convention the user asked for, a mistake not to repeat): save it to
auto memory.
   - Nothing worth recording is a valid outcome. Do not write filler.
7. Tick the slice in docs/PLAN.md. Commit everything as `slice N: <goal>`.
8. Stop. Do not start the next slice. Reply with three lines: what changed, what
was verified, any note recorded.
```

How a skill works. The frontmatter `name` becomes the `/next-slice` command. The `description` is what the model reads on every turn to decide whether the skill applies; the body is loaded only when the skill is invoked. Rule 8 ("stop") is what makes the loop reviewable: one slice, one commit, one review.

Step 31 ⌨️ Create the hook script

Save as `.claude/hooks/plan-status.sh`. It is a Bash script on both systems: on Windows, Claude Code runs hooks through Git Bash.

```
#!/usr/bin/env bash
# Stop hook: print slice progress from docs/PLAN.md after every response.
PLAN="$CLAUDE_PROJECT_DIR/docs/PLAN.md"
[ -f "$PLAN" ] || exit 0

DONE=$(grep -c '^- \[x\] \*\*Slice' "$PLAN")
TOTAL=$(grep -c '^- \[[ x]\] \*\*Slice' "$PLAN")
NEXT=$(grep -m1 '^- \[ \] \*\*Slice' "$PLAN" | sed -E 's/^- \[ \] \*\*(Slice [0-9]+): ([^*]+)\*\*.*/\1 — \2/')

if [ "$DONE" -eq "$TOTAL" ]; then
  MSG="PLAN.md: all $TOTAL slices done."
else
  MSG="PLAN.md: $DONE/$TOTAL slices done · next: $NEXT"
fi

jq -n --arg msg "$MSG" '{systemMessage: $msg}'
exit 0
```

Make it executable and test it by hand.

macOS:

```
chmod +x .claude/hooks/plan-status.sh
echo '{}' | .claude/hooks/plan-status.sh
```

PowerShell (Windows has no execute bit, so there is no `chmod`; test through Git Bash):

```
& "C:\Program Files\Git\bin\bash.exe" -c 'CLAUDE_PROJECT_DIR="$PWD" ; echo "{}" | .claude/hooks/plan-status.sh'
```

The execute bit matters for teammates on macOS who clone your repository; Step 33 records it in Git from Windows.

What you should see. `{"systemMessage":"PLAN.md: 0/8 slices done · next: Slice 1 — ..."}`. If PowerShell prints `jq: command not found`, open a new terminal (or install `jq` into a folder on `PATH`, which Git Bash shares with Windows).

How a hook works. Claude Code runs the command at the named event (`Stop` = after each response) and passes the event as JSON on stdin. A hook that prints `{"systemMessage": ...}` shows the message to you; exit code 2 on a `PreToolUse` hook blocks the tool call (you will use that on Day 2).

Step 32 ⌨️ Register the hook in `.claude/settings.json`

Replace the file with:

```
{
  "permissions": {
    "allow": ["Edit", "Bash(pytest *)", "Bash(uv run pytest *)", "Bash(uv run ruff
*)", "Bash(git add *)", "Bash(git commit *)"],
    "ask": ["Bash(git push *)"],
    "deny": ["Read(./.env)"]
  },
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/plan-status.sh"
          }
        ]
      }
    ]
  }
}
```

Add to `.gitignore`:

```
.claude/settings.local.json
```

Step 33 ▶ Commit and push

```
!git add .claude .gitignore
!git commit -m "next-slice skill and plan-status hook"
!git push
```

Windows only: between the `git add` and the `git commit`, record the execute bit for the script in Git, since the Windows file system cannot store it.

```
!git update-index --chmod=+x .claude/hooks/plan-status.sh
```

Step 34 ▶ Verify the hook and the skill are active

```
/hooks
```

What you should see. `plan-status.sh` listed under **Stop**. Hooks in `settings.json` are read when the session starts; if it is missing, `/clear` or restart.

```
/reload-skills
/
```

What you should see. `next-slice` in the command menu with its description.

```
/context all
```

What you should see. `CLAUDE.md` under Memory files, the `next-slice` description under Skills, and the token cost of each.

Part 6 — Implementing the slices

Step 35 ▶ Run the first slice

```
/clear
/next-slice
```

Watch what happens: the skill states the slice, reads the files, writes code and tests, runs the suite, ticks the box, commits, and stops with three lines. After the response the Stop hook prints `PLAN.md: 1/8 slices done · next: ...`.

Review before continuing:

```
!git show --stat HEAD
!git diff HEAD~1 -- docs/PLAN.md
```

If something is wrong, say so in plain language and let it fix and amend. Then `/clear` and `/next-slice` again.

Step 36 ▶ Ask a side question with `/btw`

While a long slice runs you can ask a question without interrupting the task or adding to its context:

```
/btw what is the current usage and context size
```

`/btw` answers in a side conversation. Use it for "what does this error mean" or "how far along are we" during long turns.

Step 37 ⌨ Add a slice-reviewer subagent

A subagent is a separate Claude with its own context, system prompt and tool list. A reviewer that cannot edit files and has not seen the implementation conversation gives you a second opinion that is not grading its own work.

Save as `.claude/agents/slice-reviewer.md`:

```
---
name: slice-reviewer
description: Reviews the most recent slice commit against docs/SPEC.md and
docs/PLAN.md. Read-only. Use after a slice is implemented, before moving on.
tools: Read, Grep, Glob, Bash(git diff *), Bash(git log *), Bash(git show *),
Bash(uv run pytest *)
model: sonnet
---

You review one slice. Do not edit anything.

1. Run `git log -1 --stat` and `git show HEAD` to see what changed.
2. Read the slice's entry in docs/PLAN.md and the relevant sections of docs/SPEC.md
and CLAUDE.md.
3. Check, in this order: (a) does the change do what the slice says, no more and no
less; (b) are the guardrails in CLAUDE.md respected (read-only SQL, limits, no
secrets); (c) do the tests cover the slice's verification method and would they
fail if the feature broke; (d) anything that will make a later slice harder.
4. Run `uv run pytest -q` and report the result.
5. Reply with: VERDICT (APPROVE or REWORK), then at most five findings, each as
`file:line — problem — suggested fix`. No praise, no summary of the diff.
```

Step 38 ▶ Run the subagent

```
/reload-skills
Run the slice-reviewer on the last commit.
```

Claude Code delegates to the subagent and returns its verdict. If the verdict is REWORK, paste the findings into a normal turn ("fix findings 1 and 3, explain why 2 is not a problem") and re-run the reviewer. Make this the habit after every slice.

Step 39 ▶ Design the UI

```
/design Check docs/PLAN.md and docs/SPEC.md and create a mockup UI for the app: login, the chat
page with database selector, question box, SQL and result panels, and the admin page for adding a
database.
```

`/design` produces a visual mockup you can react to. Iterate in plain language until the layout is right for the users in your one-pager.

Step 40 ▶ Record the design in the plan

```
Add a "UI design" section to docs/PLAN.md describing the agreed mockup: pages, components on
each, and which slice implements each page. Keep the slice checklist unchanged.
```

Why. The UI slices will read `PLAN.md`; the design has to be where they will look.

Step 41 ▶ Run several slices with `/goal`

`/goal` keeps Claude Code working, turn after turn, until a condition is met. Combined with the skill it runs the loop unattended:

```
/clear
/goal complete the slices 2 – 4 from docs/PLAN.md using /next-slice for each slice
```

The commits in the `allow` list run without prompts; a push would stop and ask. Note that the context is **not** cleared between slices under `/goal`; auto-compaction will kick in when it fills. For a fresh context per slice, run `/next-slice` by hand with `/clear` in between, or run a loop from your shell, where every `claude -p` call is a brand-new session:

macOS:

```
for i in 1 2 3; do claude -p "/next-slice" --permission-mode acceptEdits || break;
done
```

PowerShell:

```
foreach ($i in 1..3) { claude -p "/next-slice" --permission-mode acceptEdits; if
($LASTEXITCODE -ne 0) { break } }
```

When it stops, review:

```
!git log --oneline -5
Run the slice-reviewer on each of the last three commits, one at a time.
```

Step 42 ▶ Finish the plan

```
/goal complete the slices 5 – 8 from docs/PLAN.md using /next-slice for each slice
```

(Adjust the numbers to your plan.) Review as before. The Stop hook should end with `PLAN.md: all 8 slices done.`

Step 43 ▶ Export the conversation

```
/export slices_2_8.txt
```

Why. The transcript is the record of what was decided and why. Keep it with the project (outside Git, or under `docs/transcripts/`) for the retrospective at the end of the day.

Part 7 — Run the application

Step 44 ▶ Write the local deployment document

```
Write docs/RUN_LOCAL.md: prerequisites, environment variables (ASKDB_CONFIG_DIR,
ANTHROPIC_API_KEY), how to create the first admin user, how to start the API and the front end,
how to add the CloudMetrics database through the admin page, and how to run the evaluation set.
Verify every command by running it.
```

Step 45 ⌨ Start the server and the client

You will run two processes in two terminals. Open a third one for checks.

Terminal	Process	Address
A	API server (FastAPI, uvicorn)	http://127.0.0.1:8000
B	Web UI (React dev server, Vite)	http://localhost:5173

Do not expose these ports beyond your laptop. Run every command from the repository root unless a step says otherwise. Each sub-step gives the command for the **macOS Terminal** (also Linux and WSL) and for **Windows PowerShell**. Folder and module names below are the ones the course build produces; if your slices named them differently, `docs/RUN_LOCAL.md` has yours.

45.1 Open the repository

macOS:

```
cd ~/path/to/askdb
ls pyproject.toml web/package.json config/settings.yaml
```

PowerShell:

```
cd C:\path\to\askdb
Get-Item pyproject.toml, web\package.json, config\settings.yaml
```

Expected: all three paths are listed with no error. If one is missing you are in the wrong folder or a slice did not finish.

45.2 Check the dependencies

Same commands on both systems:

```
uv --version      # any recent version
node --version    # v18 or newer
npm --version     # any
```

If `uv` is missing: macOS `brew install uv`; Windows `winget install astral-sh.uv` (or the installer at https://docs.astral.sh/uv/). If Node is missing, install Node 18 or newer from https://nodejs.org (Windows: `winget install OpenJS.NodeJS.LTS`). Open a new terminal after installing.

`uv` installs Python 3.10 or newer for you on the first `uv sync`; you do not need to install Python yourself.

45.3 Install the Python packages

```
uv sync
```

Expected: the last line reads `Installed N packages` or `Audited N packages`, with no error.

45.4 Install the web packages

macOS:

```
cd web && npm install && cd ..
```

PowerShell:

```
cd web; npm install; cd ..
```

Expected: `added N packages`. A warning about a newer npm is harmless.

45.5 Check that the build works

macOS:

```
uv run pytest -q
cd web && npm run build && cd ..
```

PowerShell:

```
uv run pytest -q
cd web; npm run build; cd ..
```

Expected: pytest ends with `N passed, 3 skipped` (the 3 skipped tests need an API key; skipping them here is correct). The build ends with `✓ built in ...` and no errors. If pytest fails, stop and fix the failure before continuing.

45.6 Set the environment variables (terminal A)

The app does **not** read a `.env` file. Set the variables in every terminal that runs the API, before starting it. Replace the placeholder values.

macOS:

```
export ANTHROPIC_API_KEY="sk-ant-REPLACE_WITH_YOUR_KEY"
export ASKDB_CONFIG_DIR="$PWD/config"
# only if your key is not scoped to a workspace:
export ANTHROPIC_WORKSPACE_ID="wrkspc_REPLACE_WITH_YOUR_WORKSPACE_ID"
# only if you use the sample database in 45.7:
export CLOUDMETRICS_DB_PATH="$PWD/tests/cloudmetrics/cloudmetrics.db"
```

PowerShell:

```
$env:ANTHROPIC_API_KEY = "sk-ant-REPLACE_WITH_YOUR_KEY"
$env:ASKDB_CONFIG_DIR = "$PWD\config"
# only if your key is not scoped to a workspace:
$env:ANTHROPIC_WORKSPACE_ID = "wrkspc_REPLACE_WITH_YOUR_WORKSPACE_ID"
# only if you use the sample database in 45.7:
$env:CLOUDMETRICS_DB_PATH = "$PWD\tests\cloudmetrics\cloudmetrics.db"
```

Rules for these variables:

- `ANTHROPIC_API_KEY` is required to answer questions. Without it the app starts, but every question shows a configuration message. Never put the key in a file inside the repository.
- `ANTHROPIC_WORKSPACE_ID` is optional. Set it only when Anthropic reports that the key is not scoped to a workspace.
- `ASKDB_CONFIG_DIR` is where database configurations are written (`databases/<name>.yaml`).
- To keep the key out of your shell history, put the lines in a file **outside** the repository and load it: macOS `~/askdb_env.sh` then `source ~/askdb_env.sh` ; PowerShell `~\askdb_env.ps1` then `. ~\askdb_env.ps1` (if scripts are blocked, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once).

Check:

```
echo "$ASKDB_CONFIG_DIR"        # macOS
```

```
echo $env:ASKDB_CONFIG_DIR       # PowerShell
```

Expected: the full path to `config` .

45.7 Add the sample database from the command line (optional)

The usual way is the Admin page in the browser (Step 46). To start with the sample database already in place, copy its configuration:

macOS:

```
mkdir -p config/databases
cp tests/cloudmetrics/cloudmetrics.yaml config/databases/
```

PowerShell:

```
New-Item -ItemType Directory -Force config\databases | Out-Null
Copy-Item tests\cloudmetrics\cloudmetrics.yaml config\databases\
```

Expected: `config/databases/cloudmetrics.yaml` exists. The sample configuration refers to `${CLOUDMETRICS_DB_PATH}` , so the variable from 45.6 must be set.

45.8 Create the first account (terminal A)

Same command on both systems:

```
uv run python -m askdb.cli create-user --username your.name --role support
```

You are asked for a password twice; at least 8 characters.

Expected: `Created support account your.name.`

- This is the only way to create the first account; later accounts are created from the Admin screen.
- `--role analyst` creates an analyst account. Run the command once per account; a repeated name fails with "already exists".

45.9 Start the API server (terminal A)

Make sure terminal A has the variables from 45.6 and you are at the repository root.

```
uv run uvicorn askdb.main:app --host 127.0.0.1 --port 8000
```

Expected output, ending with:

```
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000
```

Leave this terminal running. Startup logs appear here; a database whose startup checks fail is logged as `database <name> disabled: ...`. The first start creates `data/app.db`.

45.10 Start the web UI (terminal B)

Open a **second** terminal. Terminal B does not need the API variables.

macOS:

```
cd ~/path/to/askdb/web
npm run dev
```

PowerShell:

```
cd C:\path\to\askdb\web
npm run dev
```

Expected output, ending with:

```
  VITE v5.x.x  ready in ... ms
  ➜  Local:   http://localhost:5173/
```

Leave this terminal running.

45.11 Check that both processes respond (third terminal)

macOS:

```
curl -s -o /dev/null -w "API without sign-in: %{http_code}\n"
http://127.0.0.1:8000/api/databases
curl -s -o /dev/null -w "UI: %{http_code}\n" http://localhost:5173/
```

PowerShell (`curl` is an alias for `Invoke-WebRequest` here, so use `curl.exe` or the native cmdlet):

```
curl.exe -s -o NUL -w "API without sign-in: %{http_code}`n"
http://127.0.0.1:8000/api/databases
curl.exe -s -o NUL -w "UI: %{http_code}`n" http://localhost:5173/
```

Expected: `API without sign-in: 401` (the API is up and requires a sign-in) and `UI: 200`.

45.12 Troubleshooting

Symptom	Cause and fix
`Address already in use` on 8000 or 5173	Another process holds the port. macOS: `lsof -i :8000` then `kill <pid>`. PowerShell: `Get-NetTCPConnection -LocalPort 8000 | Select OwningProcess` then `Stop-Process -Id <pid>`. Or start uvicorn with `--port 8001` and change the proxy target in `web/vite.config.*`.
Every question shows a configuration message	`ANTHROPIC_API_KEY` not set in terminal A, or set after uvicorn started. Stop the server (`Ctrl+C`), set it, start again.
`database cloudmetrics disabled: ...` in the API log	`CLOUDMETRICS_DB_PATH` not set or wrong. Check the path exists; on Windows use backslashes or forward slashes consistently.
`uv: command not found` / `uv is not recognized` after installing	Open a new terminal so `PATH` is reloaded.
PowerShell refuses to run `askdb_env.ps1`	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, once.
UI loads but sign-in fails	The account from 45.8 was not created, or the API is not running. Check terminal A.
`401` expected but `000` returned by curl	The API is not running or on another port. Check terminal A's last lines.

To stop: `Ctrl+C` in terminals A and B.

Step 46 — Add the database and ask questions

1. Open http://localhost:5173 and sign in with the account from 45.8.
2. Admin page: name `cloudmetrics`, URL `sqlite:///<absolute path>/tests/cloudmetrics/cloudmetrics.db` (Windows: `sqlite:///C:/path/to/askdb/tests/cloudmetrics/cloudmetrics.db`, forward slashes), upload `tests/cloudmetrics/cloudmetrics.yaml`. Save. Check that `config/databases/cloudmetrics.yaml` now exists. Skip this if you copied the file in 45.7.
3. Chat page: select **cloudmetrics** and ask:
   - *How many customers do we have?* (expect 200)
   - *What is the total MRR of active subscriptions?* (expect about 22,344)
   - *Which plan has the most active subscriptions?*
   - *Show the five customers with the most support tickets.* then *Only the ones on the Enterprise plan.* (a follow-up)
   - *Delete all invoices.* (must be refused)
   - *What is the weather today?* (must be declined as a non-data question)

4. Run the evaluation set as `RUN_LOCAL.md` describes and note the pass rate. This number is your baseline for Day 3.

Part 8 — Working with GitHub: issue → branch → PR

Step 47 — Create an issue on GitHub

On the repository page, **Issues → New issue**:

> **Title:** Report the token usage for queries in the chat window
>
> **Description:** The input and output token usage for the queries in the chat window will show how expensive the queries are. After showing the result and explanation for a query, show the cost (input and output token usage).

Step 48 ▶ List open issues from Claude Code

```
!gh issue list --state open
```

Step 49 ▶ Fix the issue end to end

```
/clear
Fix GitHub issue 1: create a branch, implement the fix, write tests, commit, and open a PR to
merge the changes to the main branch.
```

Claude Code reads the issue with `gh issue view 1`, creates a branch (`fix/issue-1-token-usage`), changes the API response and the chat panel, adds tests, commits, pushes (you approve the push) and runs `gh pr create`. The PR description it writes should reference the issue (`Closes #1`).

Step 50 — Review and merge

On GitHub, read the PR: the diff, the tests, the description. Optionally, from Claude Code:

```
/code-review
```

Merge the PR in the browser. Then:

```
!git checkout main
!git pull
```

Step 51 ⌨ Verify by hand

Restart the server, ask a question, and confirm the token counts appear under the answer. Close the loop: the issue is closed automatically by the merge.

Useful commands and prompts

Command	Use
`/clear`	Empty the context. Do it between tasks; the files carry the state.
`/context`, `/context all`	What is in the context and what it costs.
`/compact`	Summarise the context in place when a task is long but not finished.
`/model`, `/effort`, `/advisor`	Trade cost for reasoning depth.
`/plan` or `Shift+Tab`	Plan mode: think without changing anything.
`/btw <question>`	Side question without touching the main task.
`/goal <condition>`	Keep working until the condition holds.
`/design <brief>`	Visual mockup.
`/export <file>`	Save the transcript.
`/code-review`	Review the current diff or PR. Documentation: https://code.claude.com/docs/en/code-review
`/security-review`	Security pass over the current changes; run before merging anything that handles user input or SQL.
`/hooks`, `/reload-skills`	Check and refresh the extensions.
`/usage`	Plan limits and spend.
`!<command>`	Run a shell command and show the output. On Windows this runs in Git Bash, so use Unix syntax (`ls`, `cat`, forward slashes), not PowerShell cmdlets.
Prompt library	https://code.claude.com/docs/en/prompt-library
Best practices	https://code.claude.com/docs/en/best-practices

macOS ↔ PowerShell quick reference

For commands you type in your own terminal (not inside Claude Code):

Task	macOS Terminal	Windows PowerShell
Make folders	`mkdir -p a/b c`	`New-Item -ItemType Directory -Force a\b, c`
Copy files	`cp src/* dest/`	`Copy-Item src\* dest\`
List / show a file	`ls`, `cat file`	`ls` (alias), `Get-Content file` or `cat file` (alias)
Set an environment variable (this terminal)	`export NAME="value"`	`$env:NAME = "value"`
Read it back	`echo "$NAME"`	`echo $env:NAME`
Current folder in a value	`"$PWD/config"`	`"$PWD\config"`
Load variables from a file	`source ~/askdb_env.sh`	`. ~\askdb_env.ps1`
Chain commands	`a && b` (stop on failure)	`a; b` (always continues; check `$LASTEXITCODE`)
Make a script executable	`chmod +x script.sh`	not needed locally; `git update-index --chmod=+x script.sh` for the repository
Run a Bash script	`./script.sh`	`& "C:\Program Files\Git\bin\bash.exe" script.sh`
HTTP check	`curl -s -o /dev/null -w "%{http_code}" URL`	`curl.exe -s -o NUL -w "%{http_code}" URL`
Find who holds a port	`lsof -i :8000`	`Get-NetTCPConnection -LocalPort 8000`
Stop a process	`kill <pid>`	`Stop-Process -Id <pid>`
Home folder	`~`	`~` or `$HOME`
Path separators	`/`	`\` in PowerShell; `/` also accepted by Python, Node, Git and SQLite URLs

Habits that made the day work

- **One task, one context.** `/clear` between tasks; the files (`SPEC.md`, `PLAN.md`, `CLAUDE.md`) carry the state, not the conversation.
- **Interview, do not dictate.** Both documents were produced by answering questions; the model stayed on the problem.
- **Thin vertical slices, one commit each.** Small enough to review, large enough to test end to end.
- **Verification is part of the slice.** The skill will not tick a box without a passing suite; the reviewer will not approve without reading the diff.
- **Permissions are the safety net.** Allow the routine, ask for the irreversible, deny the secret.

Appendix A — Reference `CLAUDE.md`

```
# Ask DB

Web app in which business users ask questions of a configured database in plain English
and get a read-only SQL query, its result and a short explanation.

## Stack and commands
- Python 3.11, FastAPI, SQLAlchemy, anthropic SDK; `uv` for packages. Front end: React (Vite) in `frontend/`.
- Install: `uv sync` · Run API: `uv run uvicorn askdb.main:app --port 8010` · Front end: `cd frontend && npm run dev`
- Test: `uv run pytest -q` · Lint: `uv run ruff check .`
- Config dir: `ASKDB_CONFIG_DIR` (default `./config`); databases in `databases/<name>.yaml`.

## Layout
- `src/askdb/` application code · `tests/` pytest · `docs/` SPEC.md (source of truth), PLAN.md (slices), RUN_LOCAL.md
- `test/` sample database `cloudmetrics.db`, schema `cloudmetrics.yaml`, eval set `cloudmetrics_eval.yaml`

## Guardrails
- Only `SELECT` / `WITH` statements ever reach a database. Reject everything else before execution.
- Every query runs with a row cap and a timeout (defaults in SPEC.md).
- Never read `.env`; secrets come from environment variables only. Never commit keys.
- Database configurations are validated before they are written; a failed validation writes nothing.
- The model receives the schema YAML and its business rules; it never receives raw table data beyond query results.

## Working rules
- `docs/SPEC.md` is the source of truth. If code and spec disagree, raise it; do not silently pick one.
- `docs/PLAN.md` tracks progress: one checkbox per slice. Tick a slice only when its verification passes.
- Every change ships with tests. Do not weaken or skip a test to get green.
- Commit messages: `slice N: <goal>` for plan work; imperative mood otherwise.
```

Appendix B — Files created today

```
askdb/
├── CLAUDE.md
├── .claude/
│   ├── settings.json              # permissions + Stop hook
│   ├── skills/next-slice/SKILL.md
│   ├── hooks/plan-status.sh
│   └── agents/slice-reviewer.md
├── docs/
│   ├── one-pager.md
│   ├── SPEC.md
│   ├── PLAN.md
│   └── RUN_LOCAL.md
├── test/
│   ├── cloudmetrics.db
│   ├── cloudmetrics.yaml
│   └── cloudmetrics_eval.yaml
├── src/askdb/ · tests/ · frontend/   # produced by the slices
├── .env.example
└── .gitignore
```

Day 2 starts from this repository: the skills you write there generate `cloudmetrics.yaml`-style configuration and evaluation files for any new database, and the hooks and plugin package what you built today for a team.